

GUIÓN DE PRESENTACIÓN

Hybrid Cloud — Diagnóstico multi-capa de un pipeline de ML sobre VPN site-to-site

Valentín Rodríguez · Martina Caetano · Mikaela Maldonado · Alexia Aurrecochea

Redes de Datos · UCU 2026

Convenciones:

Nombre en azul: quién habla

Texto en cursiva: acción escénica

Texto normal: lo que se dice

SLIDE 1 — Portada

(nadie habla)

SLIDE 2 — El problema

Vale:

Buenas, somos el Equipo 1. El trabajo parte de una pregunta concreta: ¿cómo se diagnostican fallas de red sutiles en un escenario de nube híbrida?

Marti:

Una arquitectura Hybrid Cloud combina infraestructura propia con nube pública. El desafío no está en cada extremo por separado, sino en la interconexión — ahí es donde aparecen los problemas que son difíciles de ver y de correlacionar.

SLIDE 3 — Arquitectura

Mika:

La maqueta tiene tres sitios. El sitio on-premises corre en la máquina de Alexia con WSL2 — ahí está el cluster A, el pipeline de ML, y todo el stack de observabilidad. El sitio cloud-secundario es la Mac de Valentín, con el cluster B que simula el nodo remoto. Y el sitio cloud-principal es Azure Blob Storage, donde vive el dataset.

Vale:

Los dos equipos están conectados por un túnel WireGuard corriendo sobre Tailscale. Y encima de esa conectividad, Istio une los dos clusters en un único service mesh con mTLS de extremo a extremo usando una CA compartida.

SLIDE 4 — Estándares

Ale:

El núcleo técnico del trabajo gira en torno a la MTU y la fragmentación IP. El RFC 791 define el flag DF — Don't Fragment — que le dice a los routers que no fragmenten el paquete. El RFC 792 define ICMP, que es el mecanismo por el cual un router avisa al emisor que el paquete era demasiado grande. El RFC 1191 define Path MTU Discovery, que es el proceso por el cual TCP descubre la MTU del camino usando esos mensajes ICMP. El RFC 2923 documenta los problemas que aparecen cuando ese mecanismo falla — que es exactamente el PMTU black hole que investigamos. Y el RFC 4821 propone una solución alternativa que no depende de ICMP. Los dos últimos son de referencia comparativa — BGP/MPLS e IKEv2 frente a WireGuard.

SLIDE 5 — Implementación

Mika:

La construcción siguió un orden incremental. Primero levantamos el túnel WireGuard y verificamos que los dos equipos se veían. Después creamos los dos clusters de Kubernetes con CIDRs separados. Luego generamos la CA compartida e instalamos Istio con los east-west gateways y los remote secrets. Con la infraestructura lista instalamos el observatorio y finalmente el pipeline de ML.

SLIDE 6 — Experimento 01

Marti:

El primer experimento midió el overhead del service mesh. Generamos 10 requests por segundo durante 2 minutos con k6, en tres escenarios — con mesh en condiciones normales, con mesh y WAN degradada, y sin mesh.

Vale:

El resultado más interesante es que bajo WAN degradada la latencia p95 fue de 59ms, ligeramente menor al baseline de 63ms. Eso se explica por el locality load balancing de Istio — cuando el enlace WAN se degrada, el mesh dirige el tráfico preferentemente al endpoint local. Y sin mesh, 100% de error — sin mesh no hay conectividad cross-cluster.

SLIDE 7 — Experimento 02

Vale:

El segundo experimento corrió el pipeline de ML real. Se conectó a Azure Blob Storage, descargó 28.4 MB del dataset CIC-IDS2017 a través del túnel WireGuard a 2.6 MB/s, procesó 225 mil muestras, y entrenó un clasificador Random Forest en 2.7 segundos con 99.96% de accuracy. El punto no es el modelo en sí, sino demostrar que el enlace híbrido soporta cargas de trabajo reales.

SLIDE 8 — Experimento 03

Ale:

El tercer experimento es el más importante para la pregunta del trabajo. Redujimos la MTU del túnel a 576 bytes y bloqueamos los mensajes ICMP Fragmentation Needed con iptables, simulando lo que pasa en redes reales con firewalls restrictivos.

Marti:

El síntoma del PMTU black hole es que la conexión se congela sin dar error explícito. TCP queda esperando ACKs que nunca llegan. En el observatorio se ve capa por capa — en capa 3 el RTT subió a picos de 300ms, en capa 4 aparecieron retransmisiones TCP, en capa 7 la latencia HTTP se disparó.

Ale:

La solución es MSS clamping con iptables — forzar que TCP negocie un segmento compatible con la MTU del túnel sin depender de ICMP. Después del fix, las retransmisiones cayeron a cero en el siguiente scrape de Prometheus, sin reiniciar nada.

SLIDE 9 — Demo en vivo

Vale:

Ahora les mostramos todo corriendo en tiempo real. El Acto 1 es el estado normal — el pipeline de ML ejecutándose, el observatorio con latencias bajas, cero retransmisiones, y en Kiali el grafo del service mesh con los candados de mTLS.

Ale:

El Acto 2 es el diagnóstico en vivo. Inducimos el PMTU black hole, lo seguimos capa por capa en Grafana, y aplicamos el MSS clamping para ver la recuperación inmediata.

(arrancan la demo)

SLIDE 10 — Discusión

Mika:

Cuatro conclusiones. El overhead del mesh es bajo — 63ms p95 con 0% de error. El mesh aísla la degradación WAN gracias al locality load balancing. El mesh no es solo overhead, es infraestructura de conectividad — sin él no hay cross-cluster. Y el observatorio multi-capa fue lo que permitió correlacionar el síntoma en capa 7 con la causa raíz en capa 3.

SLIDE 11 — Limitaciones

Marti:

Tres limitaciones honestas. WireGuard maneja la fragmentación internamente sobre UDP, así que reproducir un PMTU black hole clásico requirió forzar el escenario. tc netem sobre WSL no propaga la degradación con total fidelidad. Y son 10 req/s — es una maqueta de diagnóstico, no una prueba de carga de producción.

SLIDE 12 — Referencias

(se muestra, nadie la comenta)

SLIDE 13 — Cierre

Vale:

Eso es todo. Construimos una maqueta funcional que integra LAN/WAN, VPN, SDN, APIs y mTLS de punta a punta, corrimos un pipeline de ML real sobre el enlace híbrido, y diagnosticamos en vivo un PMTU black hole con el observatorio multi-capa.

Ale:

El repositorio está en github.com/aleaurre/redato-hybrid-cloud. Abrimos a preguntas.
